

Queries

☐ Checking the total number of transactions

```
SELECT COUNT(*) AS total_transactions FROM transactions;
```

Results		Messages	
	total_transactions		
1	1000000		

☐ Calculating the total revenue generated from all transactions

```
SELECT SUM(total_cost) AS total_revenue FROM transactions;
```

Results		Messages	
	total_revenue		
1	52455220.4000007		

☐ Finding out how many unique customers have made purchases

```
SELECT COUNT(DISTINCT customer_name) AS unique_customers FROM transactions;
```

	unique_customers
1	329738

☐ Identifying the top 10 highest-spending customers

```
SELECT TOP 10 customer_name, SUM(total_cost) AS total_spent  
FROM transactions  
GROUP BY customer_name  
ORDER BY total_spent DESC;
```

	customer_name	total_spent
1	Michael Smith	23768.53
2	James Smith	18464.04
3	Michael Johnson	18032.17
4	Christopher Smith	17006.64
5	David Smith	16547.27
6	Michael Williams	15353.61
7	Michael Jones	15164.54
8	Jennifer Smith	14465.09
9	John Smith	14285.09
10	Michael Brown	14143.08

☐ **Calculating the average value of a single transaction**

```
SELECT AVG(total_cost) AS avg_order_value FROM transactions;
```

	avg_order_value
1	52.4552204000007

☐ **Understanding customer payment preferences by counting the usage of each payment method**

```
SELECT payment_method, COUNT(*) AS usage_count  
FROM transactions  
GROUP BY payment_method  
ORDER BY usage_count DESC;
```

	payment_method	usage_count
1	Cash	250230
2	Debit Card	250074
3	Credit Card	249985
4	Mobile Payment	249711

☐ **Checking which cities contribute the most to total sales**

```
SELECT city, SUM(total_cost) AS total_sales  
FROM transactions  
GROUP BY city  
ORDER BY total_sales DESC;
```

	city	total_sales
1	Dallas	5277111.53000003
2	Boston	5263307.95999999
3	Chicago	5263187.45
4	New York	5252469.91999992
5	Houston	5247054.78000001
6	San Francisco	5241099.85999995
7	Miami	5240498.44000002
8	Seattle	5235365.43000006
9	Los Angeles	5232393.19
10	Atlanta	5202731.83999991

- ☐ **Identifying customers who have only made a single purchase, which could indicate a churn risk**

```
SELECT customer_name, COUNT(*) AS order_count
FROM transactions
GROUP BY customer_name
HAVING COUNT(*) = 1;
```

	customer_name	order_count
1	Derek David	1
2	Philip Lamb	1
3	Belinda Bush	1
4	Michaela Navarro	1
5	Bob Armstrong	1
6	Selena Wilson	1
7	Barry Mack	1
8	Erica Li	1
9	Gail Wiley	1
10	Michael Richmond Jr.	1
11	Anne Warren	1
12	Andrew Hull	1
13	Ricky Whitehead	1
14	Sonya Powers	1
15	Rebecca Odom	1
16	Jeremiah Arias	1
17	April Cardenas	1
18	Brenda Holland MD	1
19	Penny Sanford	1
20	Miguel Montoya	1

- ☐ **Determining the last purchase date for each customer**

```
SELECT customer_name, MAX(date) AS last_purchase_date
FROM transactions
GROUP BY customer_name;
```

	customer_name	last_purchase_date
1	David Daniel	2023-04-18 00:29:35.0000000
2	Crystal Quinn	2020-08-04 05:45:14.0000000
3	Renee Freeman	2022-03-23 10:22:43.0000000
4	George Diaz	2022-11-29 08:03:17.0000000
5	Stephanie Watkins	2022-09-18 18:36:31.0000000
6	Jamie Cameron	2021-02-04 13:04:10.0000000
7	Dustin Jordan	2022-05-24 21:44:49.0000000
8	Jodi Ali	2023-10-03 20:30:16.0000000
9	Patrick Terry	2024-05-03 19:54:26.0000000
10	Ariel Huerta	2023-10-19 00:43:46.0000000
11	Tanner Adams	2024-02-29 11:06:53.0000000
12	Mr. Geoffrey Delgado	2022-01-29 16:38:32.0000000
13	Jimmy Stone	2024-03-19 23:21:43.0000000
14	Kevin Bates	2024-04-11 00:24:53.0000000
15	Randy Medina	2024-04-08 23:18:57.0000000
16	Chad Walker	2024-04-14 04:04:59.0000000
17	Ernest Myers	2020-06-25 09:00:26.0000000
18	Kevin Yu	2021-09-03 11:52:25.0000000
19	Paul Werner	2021-09-10 01:18:33.0000000
20	Krista Stevenson	2022-09-26 01:54:34.0000000

☐ **Finding the top 10 most frequently purchased products**

```
SELECT TOP 10 product, COUNT(*) AS times_purchased
FROM transactions
GROUP BY product
ORDER BY times_purchased DESC;
```

	product	times_purchased
1	[Toothpaste]	4893
2	[Deodorant]	2541
3	[Honey]	2540
4	[Eggs]	2515
5	[Olive Oil]	2505
6	[Vinegar]	2505
7	[Cleaning Spray]	2503
8	[Cereal Bars]	2494
9	[Pasta]	2493
10	[Cleaning Rags]	2492

☐ **Identifying the product that has generated the highest total revenue**

```
SELECT TOP 1 product, SUM(total_cost) AS total_revenue
FROM transactions
GROUP BY product
ORDER BY total_revenue DESC;
```

	product	total_revenue
1	[Toothpaste]	253689.01

☐ **Measuring how often customers place repeat orders by calculating the average time between their purchases**

```
WITH purchase_gaps AS (
  SELECT
    customer_name,
    date,
    LAG(date) OVER (PARTITION BY customer_name ORDER BY date) AS
prev_purchase
  FROM transactions
)
SELECT
  customer_name,
  AVG(DATEDIFF(DAY, prev_purchase, date)) AS avg_days_between_orders
FROM purchase_gaps
WHERE prev_purchase IS NOT NULL
GROUP BY customer_name
ORDER BY avg_days_between_orders DESC;
```

	customer_name	avg_days_betwe
1	Pamela Hammond	1598
2	Daisy Dickerson	1595
3	Eric Christian	1594
4	Sabrina Perez	1588
5	Tamara Cuevas	1583
6	Jesse Dougherty	1579
7	Diana Huang	1578
8	Paige Butler	1578
9	Theresa Tucker	1578
10	Devin Lowe	1574
11	Brandy Lozano	1574
12	Carol Whitehead	1574
13	Katrina Singleton	1569
14	Tyler Frank	1569
15	Karen Werner	1567
16	Ruth Andrews	1567
17	Tracy Maldonado	1565
18	Leslie Burnett	1564
19	Melinda Gould	1564
20	Andrea Levy	1564

☐ **Identifying customers whose spending has increased over time**

```

WITH spending_growth AS (
  SELECT
    customer_name,
    year,
    SUM(total_cost) AS yearly_spent
  FROM transactions
  GROUP BY customer_name, year
)
SELECT
  customer_name,
  MIN(yearly_spent) AS min_spent,
  MAX(yearly_spent) AS max_spent
FROM spending_growth
GROUP BY customer_name
HAVING MAX(yearly_spent) > MIN(yearly_spent)
ORDER BY max_spent DESC;

```

	customer_name	min_spent	max_spent
1	Michael Smith	2422.07	5735.81
2	Michael Johnson	1540.97	4960.41
3	David Smith	2173.9	4665.26
4	James Smith	1390.3	4473.16
5	Michael Williams	1289.12	3967.18
6	Christopher Smith	1721.93	3949.7
7	John Smith	916.81	3927.96
8	Michael Jones	1006.39	3909.43
9	Jennifer Smith	862.14	3890.79
10	Matthew Smith	902.83	3711.48
11	Jennifer Johnson	937.65	3707.13
12	Robert Smith	1066.81	3534.28
13	Michael Brown	1655.89	3531.18
14	Christopher Johnson	1278.11	3489.06
15	William Smith	935.66	3427.31
16	David Johnson	1412.33	3419.51
17	James Johnson	1424.99	3405.82
18	Robert Johnson	1110.02	3235.47
19	Michael Miller	883.44	3151.88
20	Jennifer Brown	202.04	3054.14

☐ Finding the highest-spending customer in each city

```

WITH city_top_spenders AS (
  SELECT
    city,
    customer_name,
    SUM(total_cost) AS total_spent,
    RANK() OVER (PARTITION BY city ORDER BY SUM(total_cost) DESC) AS rank
  FROM transactions
  GROUP BY city, customer_name
)
SELECT city, customer_name, total_spent
FROM city_top_spenders
WHERE rank = 1;

```

	city	customer_name	total_spent
1	Atlanta	Michael Smith	2921
2	Boston	Michael Johnson	1884.95
3	Chicago	James Smith	2225.09
4	Dallas	Michael Smith	2567.87
5	Houston	James Smith	1980.57
6	Los Angeles	Michael Johnson	2281.32
7	Miami	Michael Smith	2348.52
8	New York	Michael Smith	2871.73
9	San Francisco	Michael Smith	2886.17
10	Seattle	Michael Smith	2553.07

☐ **Estimating the customer lifetime value (CLV) for the top 10 customers**

```
SELECT TOP 10
    customer_name,
    COUNT(*) AS total_orders,
    AVG(total_cost) AS avg_order_value,
    (COUNT(*) * AVG(total_cost)) AS estimated_clv
FROM transactions
GROUP BY customer_name
ORDER BY estimated_clv DESC;
```

	customer_name	total_orders	avg_order_value	estimated_clv
1	Michael Smith	454	52.35359030837	23768.53
2	James Smith	337	54.7894362017805	18464.04
3	Michael Johnson	341	52.8802639296188	18032.17
4	Christopher Smith	301	56.500465116279	17006.64
5	David Smith	309	53.5510355987055	16547.27
6	Michael Williams	304	50.5052960526315	15353.61
7	Michael Jones	285	53.2089122807017	15164.54
8	Jennifer Smith	269	53.7735687732342	14465.09
9	John Smith	288	49.6010069444445	14285.09
10	Michael Brown	274	51.6170802919709	14143.08

☐ **Ranking store types based on total revenue generated**

```
SELECT store_type, SUM(total_cost) AS total_revenue
FROM transactions
GROUP BY store_type
ORDER BY total_revenue DESC;
```

	store_type	total_revenue
1	Pharmacy	8766679.00999998
2	Supermarket	8763455.20999998
3	Warehouse Club	8760029.02999991
4	Convenience Store	8731901.35999996
5	Department Store	8731555.57000009
6	Specialty Store	8701600.21999995

☐ **Checking if returning customers are more likely to use promotions**

```
SELECT promotion, COUNT(*) AS times_used
FROM transactions
WHERE customer_name IN (
    SELECT customer_name FROM transactions GROUP BY customer_name HAVING
    COUNT(*) > 1
)
GROUP BY promotion
ORDER BY times_used DESC;
```

	promotion	times_used
1	Discount on Selected Items	274638
2	None	274588
3	BOGO (Buy One Get One)	273915

☐ Analyzing revenue trends over the last two years to spot any seasonal sales patterns

```
SELECT year, month, SUM(total_cost) AS monthly_revenue
FROM transactions
GROUP BY year, month
ORDER BY year DESC, month DESC;
```

	year	month	monthly_revenue
1	2024	5	586965.490000003
2	2024	4	976736.250000006
3	2024	3	1019427.01
4	2024	2	943285.440000001
5	2024	1	1013783.66
6	2023	9	985031.040000005
7	2023	8	1015234.149999999
8	2023	7	1029246.18
9	2023	6	965819.819999997
10	2023	5	1014930.61
11	2023	4	978488.790000003
12	2023	3	1033556.59
13	2023	2	926532.969999997
14	2023	12	1017350.68
15	2023	11	989565.159999994
16	2023	10	1019665.03
17	2023	1	1008107.16
18	2022	9	977087.510000005
19	2022	8	1006269.539999999
20	2022	7	1016872.190000001

☐ Measuring how frequently discounts are used in transactions

```
-- Measuring how frequently discounts are used in transactions
SELECT COUNT(*) AS discounted_orders,
       COUNT(*) * 100.0 / (SELECT COUNT(*) FROM transactions) AS
discount_usage_percentage
FROM transactions
WHERE discount_applied = 1;
```

	discounted_orders	discount_usage_percentage
1	500104	50.010400000000

☐ **Identifying customers who primarily buy only when there's a discount**

```
SELECT customer_name,
       COUNT(*) AS total_orders,
       SUM(CASE WHEN discount_applied = 1 THEN 1 ELSE 0 END) AS discounted_orders,
       SUM(CASE WHEN discount_applied = 1 THEN 1 ELSE 0 END) * 100.0 / COUNT(*)
AS discount_dependency
FROM transactions
GROUP BY customer_name
HAVING (SUM(CASE WHEN discount_applied = 1 THEN 1 ELSE 0 END) * 100.0 /
COUNT(*)) > 80 -- More than 80% of purchases were discounted
ORDER BY discount_dependency DESC;
```

	customer_name	total_orders	discounted_orders	discount_dependency
1	Amber Massey	1	1	100.000000000000
2	Arthur French	1	1	100.000000000000
3	Bradley David	1	1	100.000000000000
4	Brianna Ortega	2	2	100.000000000000
5	Joshua Patton	4	4	100.000000000000
6	Jermaine David	1	1	100.000000000000
7	Julian McKenzie	1	1	100.000000000000
8	Dr. Ashley Moses DDS	1	1	100.000000000000
9	Ms. Terry Adams	1	1	100.000000000000
10	Martin Meyer	1	1	100.000000000000
11	Mr. Brian Martinez	2	2	100.000000000000
12	Vincent Rice	2	2	100.000000000000
13	Mr. Connor Hopkins	1	1	100.000000000000
14	Kathleen Sharp	1	1	100.000000000000
15	Julie Mack DDS	1	1	100.000000000000
16	Barbara Nolan	1	1	100.000000000000
17	Karina Diaz	1	1	100.000000000000
18	Sheena Clay	1	1	100.000000000000
19	Briana Cowan	1	1	100.000000000000
20	Carla Boyd	2	2	100.000000000000

☐ **Identifying the best-selling product for each month**

```
WITH ranked_sales AS (
  SELECT product, year, month, COUNT(*) AS total_sold,
         RANK() OVER (PARTITION BY year, month ORDER BY COUNT(*) DESC) AS
rank
  FROM transactions
  GROUP BY product, year, month
)
SELECT year, month, product, total_sold
FROM ranked_sales
WHERE rank = 1
ORDER BY year DESC, month DESC;
```

	year	month	product	total_sales
1	2024	5	['Toothpaste']	55
2	2024	4	['Toothpaste']	94
3	2024	3	['Toothpaste']	105
4	2024	2	['Toothpaste']	93
5	2024	1	['Toothpaste']	86
6	2023	9	['Toothpaste']	107
7	2023	8	['Toothpaste']	83
8	2023	7	['Toothpaste']	109
9	2023	6	['Toothpaste']	88
10	2023	5	['Toothpaste']	112
11	2023	4	['Toothpaste']	95
12	2023	3	['Toothpaste']	105
13	2023	2	['Toothpaste']	95
14	2023	12	['Toothpaste']	99
15	2023	11	['Toothpaste']	92
16	2023	10	['Toothpaste']	95
17	2023	1	['Toothpaste']	87
18	2022	9	['Toothpaste']	87
19	2022	8	['Toothpaste']	97
20	2022	7	['Toothpaste']	84
